

Documentation et lisibilité

GRO120 - Introduction à la programmation en robotique

François Ferland, automne 2025



Contenu

- ▶ Écrire des commentaires utiles
- ▶ Choisir les bons noms de variables et fonctions
- ▶ Organiser son code de façon à augmenter sa lisibilité



Un exemple simple (Arduino)

```
PORTB.rmode |= (1 << 4) ;
```



Un exemple simple (Arduino)

```
// Sets flag 4 to 1  
PORTB.rmode |= (1 << 4);
```



Un exemple simple (Arduino)

```
// Sets Port B pin 4 (pin D20) to  
// output mode  
PORTB.rmode |= (1 << 4);
```



“Any fool can write code that a computer can understand. Good programmers write code that humans can understand.”

Martin Fowler



Commentaires dans le code

- ▶ Documenter le **pourquoi** et non le **quoi**
- ▶ Vous pouvez assumer que vos lecteurs savent programmer, ce n'est pas un cours de programmation



Deuxième exemple (Python)

```
def ms1(array):
    l = len(array)
    if l <= 1:
        return array

    m = int(l/2)
    l1 = array[0:m]
    r1 = array[m:]

    lls = ms1(l1)
    rls = ms1(r1)

    il = 0
    ir = 0
    ls = []

    while ((il < len(lls)) and (ir < len(rls))):
        if lls[il] < rls[ir]:
            ls.append(lls[il])
            il += 1
        else:
            ls.append(rls[ir])
            ir += 1

    if (il < len(lls)):
        ls.extend(lls[il:])
    if (ir < len(rls)):
        ls.extend(rls[ir:])

    return ls
```



Deuxième exemple (Python)

```
def ms2(array):
    length = len(array)
    if length <= 1:
        return array

    middle = int(length/2)
    left_list = array[0:middle]
    right_list = array[middle:]

    left_list_sorted =
ms2(left_list)
    right_list_sorted =
ms2(right_list)

    il = 0
    ir = 0
    list_sorted = []

    while ((il < len(left_list_sorted)) and
           (ir < len(right_list_sorted))):
        if left_list_sorted[il] <
right_list_sorted[ir]:
            list_sorted.append(left_list_sorted[il])
            il += 1
        else:
            list_sorted.append(right_list_sorted[ir])
            ir += 1

    if (il < len(left_list_sorted)):
        list_sorted.extend(left_list_sorted[il:])
    if (ir < len(right_list_sorted)):
        list_sorted.extend(right_list_sorted[ir:])

    return list_sorted
```



Deuxième exemple (Python)

```
def merge_sort(array):  
    """ Returns a sorted list of the items in array  
  
    Recursively splits the array in half and merge the sub-lists in the end.  
    """  
  
    length = len(array)  
    if length <= 1:  
        return array                # Empty or single item, nothing to sort  
  
    middle = int(length/2)           # Split in halves  
    left_list  = array[0:middle]  
    right_list = array[middle:]  
  
    left_list_sorted  = ms2(left_list)  # Sort both halves  
    right_list_sorted = ms2(right_list)  
  
    # Merge back both halves  
    return merge_sorted_lists(left_list_sorted, right_list_sorted)
```



Deuxième exemple (Python)

```
def merge_sorted_lists(left_list_sorted, right_list_sorted):
    """ Takes two sorted lists, return the (sorted) merge of the two.
    """
    il = 0
    ir = 0
    list_sorted = []
    # Take the first item of each list, add the smallest to the result.
    # Stops when one of the list is empty.
    while ((il < len(left_list_sorted)) and (ir < len(right_list_sorted))):
        if left_list_sorted[il] < right_list_sorted[ir]:
            list_sorted.append(left_list_sorted[il])
            il += 1
        else:
            list_sorted.append(right_list_sorted[ir])
            ir += 1

    # Append what's left in either list. What is left is necessarily greater
    # than the last added item
    if (il < len(left_list_sorted)):
        list_sorted.extend(left_list_sorted[il:])
    if (ir < len(right_list_sorted)):
        list_sorted.extend(right_list_sorted[ir:])

    return list_sorted
```



Choisissez des noms traduisant l'intention de votre code

- ▶ `int d;`
 - ▶ On ne connaît que le type et non qu'est-ce que ça représente
- ▶ `int d; // Number of days`
 - ▶ Le commentaire est utile, mais ne suit pas la variable
- ▶ `int number_of_days;`
 - ▶ Mieux, mais qu'est-ce que représente le nombre de jours ? Une durée ? Depuis quand ?
- ▶ `int days_since_start;`
 - ▶ Idéal, on communique à la fois qu'est-ce que c'est et à quoi ça sert



Attention au contexte ...

- ▶ `double joint_angle;`
- ▶ `double joint_angular_velocity;`
- ▶ `double q;`
- ▶ `double q_dot;`

Quel ensemble préférez-vous ?

À l'intérieur d'un programme de contrôle de moteur, le contexte nous informe déjà de la signification de `q` et `q_dot`

La version plus longue des noms peut alourdir le code et nuire à la lisibilité



Évitez la désinformation

- ▶ `joints_list = 'left_arm_joints'`
`joints = ['shoulder_pan', 'shoulder_tilt']`
 - ▶ `joints_list` n'est pas une vraie liste, mais donne l'impression qu'on pourrait obtenir sa taille avec `len(joints_list)`
- ▶ `joints_list_name = 'left_arm_joints'`
`joints_list = ['shoulder_pan', 'shoulder_tilt']`



Évitez la désinformation par des commentaires désuets ou erronés

```
def is_ready():  
    """ Returns if the controller is ready or not.  
    """  
  
    wait_until(ctrl.is_ready())  
    return True
```

► Où est le problème ?

- La fonction ne retourne jamais si le contrôleur n'est pas prêt



Communiquez le fonctionnement de vos méthodes par leur nom

```
def calc_torque(with_gravity):  
    tq = f * l  
    if (with_gravity):  
        tq += g * m * l  
  
def update():  
    use_gravity = ctrl.use_gravity()  
    tq = calc_torque(use_gravity)
```



Communiquez le fonctionnement de vos méthodes par leur nom

```
def calc_torque():  
    return f * l  
  
def calc_torque_with_gravity():  
    return calc_torque() + g * m * l  
  
def update()  
    use_gravity = ctrl.use_gravity()  
  
    if (use_gravity):  
        tq = calc_torque_with_gravity()  
    else:  
        calc_torque()
```



Plutôt qu'un paramètre changeant le fonctionnement de notre méthode, on en propose une deuxième pour indiquer le comportement différent



À retenir

- ▶ **Quoi:** Votre code doit être expressif, servez-vous de lui pour exprimer ce qu'il fait.
- ▶ **Comment:** La documentation explique **comment** s'en servir et à quoi il sert
- ▶ **Pourquoi:** Les commentaires expliquent **pourquoi** votre code est écrit d'une certaine manière ce qu'il tente d'accomplir
- ▶ **Qui et quand:** Les outils de contrôle de version comme git permettent de suivre **qui** a écrit le code et **quand**. Ils permettent de situer le raisonnement dans le temps

